BDI Passenger Agents in a SimPy Charging Sandbox: Pareto Kiosk Sizing under Thunderstorm Delay

HiMCM 2019 Problem A — Team GoHiMCM

Companion files: `summary_sheet.tex` (one page), `airport_authority_letter.tex` (operations brief), `ai_disclosure.tex` (COMAP AI report). All tables and figures read locked files under `results/` and `paper_figures/`.

Contents

1 Introduction and micro-cognitive formulation **2**

2 SimPy discrete-event queueing sandbox and control-flow mapping **2**

3 FastMCP protocol and LangGraph self-reflection agent **4**

4 Dual-LLM autonomous planning and empirical Pareto sizing **4**

5 Cascading flight delay stress testing and resilience **6**

6 Strengths and limitations **7**

1 Introduction and micro-cognitive formulation

Airports sell boarding time and phone battery at the same desk. A dead phone raises missed-connection risk and service complaints. We size charging kiosks under CapEx and floor-area caps.

We model each traveler as a **BDI passenger agent** (Belief–Desire–Intention) [1]. Beliefs hold battery state of charge (SOC) and minutes to boarding T_{board}. Desires encode "stay connected until gate call." Intentions select a charging kiosk, wait, rent a power bank, or abort.

Empirical priors in our code are fixed and auditable. About 25% of arrivals start anxious about low SOC. Boarding slack is drawn as $T_{\text{board}} \sim U(45, 120)$ minutes. Utility U_{charge} rejects any plan that risks a missed flight.

The agent state machine is

$$\text{IDLE} \to \text{SEEK} \to \text{QUEUE} \to \{\text{CHARGE}, \text{RENT}, \text{RENEGING}, \text{ABORT}\}.$$

Reneging probability rises when T_{board} is short and the wait queue is long. Unit test: if $T_{\text{board}} = 10$ and queue length > 5, reneging probability equals 1.

Design questions. How many fast kiosks, slow lockers, and power-bank banks should we buy? What CapEx–wait trade-off is non-dominated? How does a 17:00 thunderstorm delay shock change reneging probability?

Caps (locked). Budget $\leq$ \$80,000. Area $\leq 40\,\text{m}^2$. A fast kiosk costs \$2,000. A slow locker costs \$1,200. A power-bank bank costs \$3,500. Unit areas are 0.8, 2.0, and $0.5\,\text{m}^2$.

2 SimPy discrete-event queueing sandbox and control-flow mapping

We embed each BDI passenger agent in a SimPy process (`src/des_env/airport_sim.py`). Three resource pools map to hardware: C_{fast} (65 W fast kiosk), C_{slow} (15 W locker), C_{bank} (power-bank rental).

Arrivals follow a non-homogeneous Poisson process over 16 hours (06:00–22:00). Intensity $\lambda(t)$ has three daily peaks. Default normal day: $\lambda_{\text{base}} = 0.35$, peak multiplier 3. Morning-rush densify runs use $\lambda_{\text{base}} = 0.55$, multiplier 4.

Figure 1 shows a Stage-2 baseline day with pack $(C_{\text{fast}}, C_{\text{slow}}, C_{\text{bank}}) = (12, 15, 2)$. Table 1 lists locked KPIs from `stage2_baseline_metrics.csv`. Mean wait time is 0.00 minutes. Reneging probability is 0.00. Fast-kiosk utilization is 5.72%. Slow-locker utilization is 24.24%. $n = 441$ arrivals; 110 anxious seekers.

Control-flow mapping. `passenger_process` requests a resource, waits, then charges or rents. Boarding deadline forces early release of the charging kiosk. Telemetry records queue length, system load, and intention flows.

Storm injection (preview). At $t = 660$ minutes (17:00), 40% of later departures gain 2–4 hours of delay. SOC drain doubles for delayed BDI passenger agents. Section 5 reports the stress KPIs.

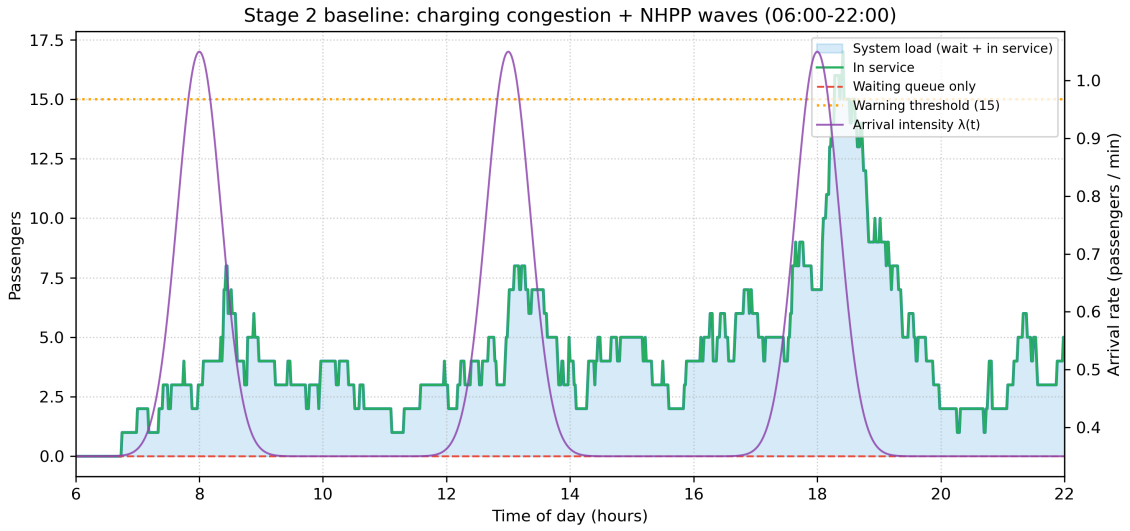


Figure 1: Stage-2 baseline load and intensity (12/15/2, normal day). Source: `paper_figures/stage2_queue_baseline.png`.

Table 1: Stage-2 baseline KPIs. Source: `stage2_baseline_metrics.csv`.

Metric	Symbol / field	Value
Arrivals	`total_passengers`	441
Anxious seekers	`anxious_passengers`	110
Served at fast kiosk	`served_fast`	37
Served at slow locker	`served_slow`	73
Power-bank rentals	`served_powerbank`	0
Mean wait (min)	`mean_wait_time`	0.00
Reneging probability	`reneging_rate`	0.00
Max system load	`max_system_load`	17
Util. fast / slow	utilization	0.057 / 0.242

3 FastMCP protocol and LangGraph self-reflection agent

Agents must call the sandbox without rewriting SimPy internals. We wrap the DES as a FastMCP server (`src/mcp_server/server.py`).

Tools. `run_charging_simulation` runs one day and returns KPIs. `validate_infrastructure_budget` checks CapEx and area caps. Resource `config://airport_constraints` publishes unit costs.

LangGraph loop. The reflection agent (`src/des_agent/reflection_agent.py`) runs

$$\text{propose} \rightarrow \text{validate} \rightarrow \text{simulate} \rightarrow \text{reflect} \rightarrow \text{check}.$$

A local Ollama model (`qwen2.5:7b`) drafts the next pack. If Ollama is offline, a deterministic rule proposer continues. Budget rejection is limited to three retries per iteration.

Acceptance targets. Mean wait ≤ 5 minutes. Reneging probability ≤ 0.05. Traces are stored under `results/reflection_*.json`.

This loop is the micro-search engine for Section 4. It does not invent CSV rows. Every accepted candidate is a real DES call.

4 Dual-LLM autonomous planning and empirical Pareto sizing

A planning agent splits the CapEx problem into four subtasks (`planning_agent.py`). Subtask budgets are \$40,000, \$80,000, pricing free-window, and pricing full-metered. Each subtask calls the reflection loop. Summary file: `autonomous_planning_summary.json` (`plan_source=ollama`).

Table 2 lists locked subtask outcomes. Pack 4/0/3 meets a low CapEx but shows reneging probability 0.637. That pack is not service-feasible for kiosk sizing.

Table 2: Autonomous planning candidates. Source: `autonomous_planning_summary.json`.

ID	Pack (C_f, C_s, C_b)	CapEx (USD)	Wait (min)	Reneging
001	8/10/3	38,500	0.163	0.000
002	15/10/8	70,000	0.163	0.000
003	4/0/3	18,500	0.000	0.637
004	20/10/8	80,000	0.163	0.000

Pareto construction. We re-simulate planning packs under rush intensity. We densify a feasible hardware grid (216 unique packs). Objectives:

$$\min \text{CapEx}, \qquad \min \overline{W}.$$

We keep only packs with reneging probability ≤ 0.05. We extract the non-dominated front (`optimal_kiosk_sizing.csv`).

Table 3 lists the seven Pareto points. The recommended knee sits under the \$80,000 CapEx line. The knee pack is $(C_{\text{fast}}, C_{\text{slow}}, C_{\text{bank}}) = (8, 16, 0)$. CapEx is \$35,200. That CapEx uses 44% of the CapEx cap. Area is $38.4\,\text{m}^2$. Rush mean wait is 0.0115 minutes. Rush reneging probability is 0.00.

The cheapest Pareto pack is 4/8/0 with wait 5.91 min. Moving to the knee cuts mean wait by 99.8%. Extra CapEx beyond \$35,200 does not reduce wait on this front.

Table 3: Pareto front (reneging $\leq 5\%$). Source: `optimal_kiosk_sizing.csv`.

C_f	C_s	C_b	CapEx	Wait (min)	Reneging
4	8	0	17,600	5.909	0.046
2	12	0	18,400	3.575	0.020
4	12	0	22,400	1.466	0.010
6	12	0	26,400	1.258	0.010
4	16	0	27,200	1.038	0.000
6	16	0	31,200	0.124	0.000
8	16	0	35,200	0.011	0.000

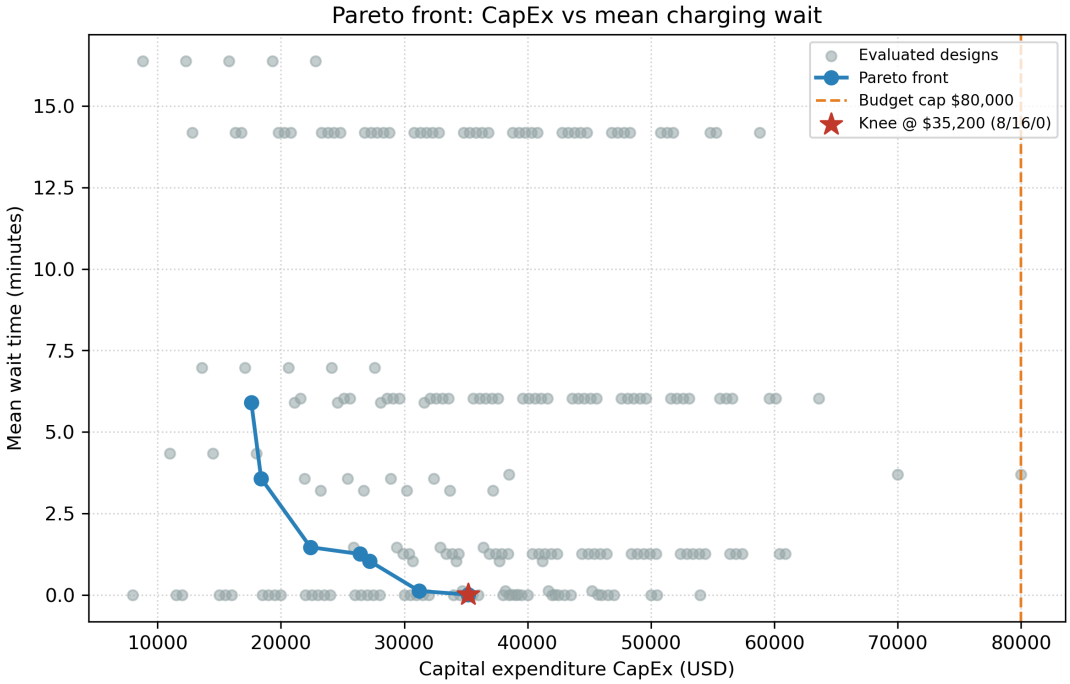


Figure 2: CapEx versus mean wait with $80,000 budget line and knee. Source: `fig_pareto_cost_vs_wait.png`.

5 Cascading flight delay stress testing and resilience

We inject a thunderstorm delay at 17:00. 40% of later departures receive 2–4 hours of added delay. Delayed BDI passenger agents double SOC drain. $n = 621$ arrivals share one random seed across packs.

We compare empirical pack 12/15/2 (CapEx \$49,000) to the Pareto knee 8/16/0 (CapEx \$35,200). Table 4 copies `thunderstorm_stress_comparison.csv`.

Table 4: Thunderstorm stress KPIs (shared seed). Source: `thunderstorm_stress_comparison.csv`.

KPI	Baseline 12/15/2	Knee 8/16/0
CapEx (USD)	49,000	35,200
Mean wait (min)	4.591	2.347
Reneging probability	0.0229	0.0114
Max wait-queue length	6	6
Max system load	23	24
Storm-delayed agents	155	165
Arrivals	621	621

Resilience deltas (locked). Mean wait falls by 48.9% (4.591 → 2.347 min). Reneging probability falls by 50.3% (0.0229 → 0.0114). CapEx falls by 28.2% versus the empirical pack. Peak wait-queue length stays at 6 for both packs.

Figure 3 plots 16-hour load with a red storm window. Figure 4 shows BDI intention splits under the same shock. Anxious seekers divert among fast kiosk, slow locker, power bank, and abort.

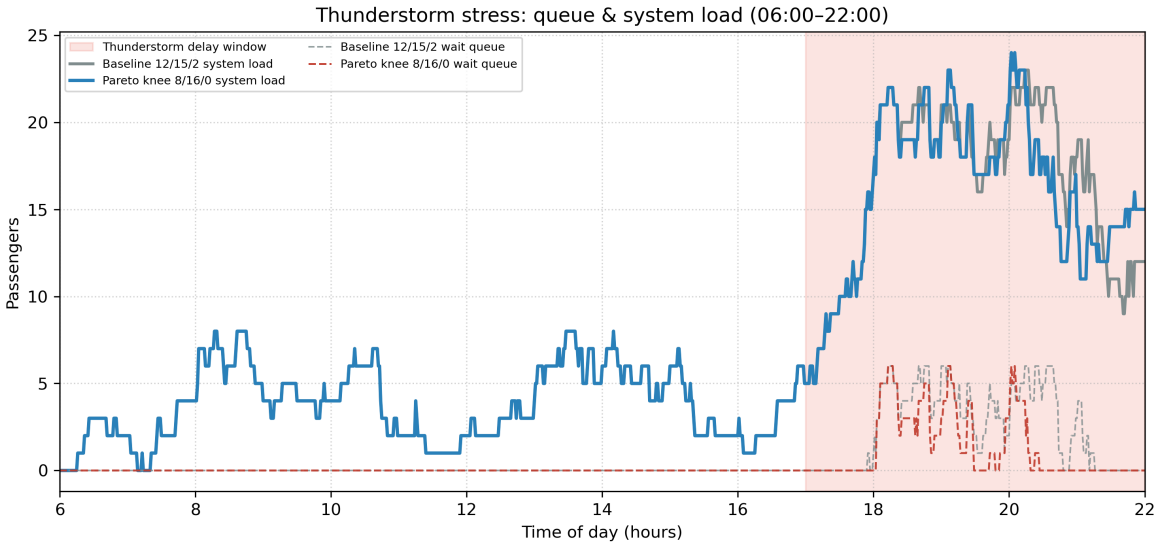


Figure 3: System load and wait queue under thunderstorm delay. Source: `fig_queue_length_delay_stress.png`.

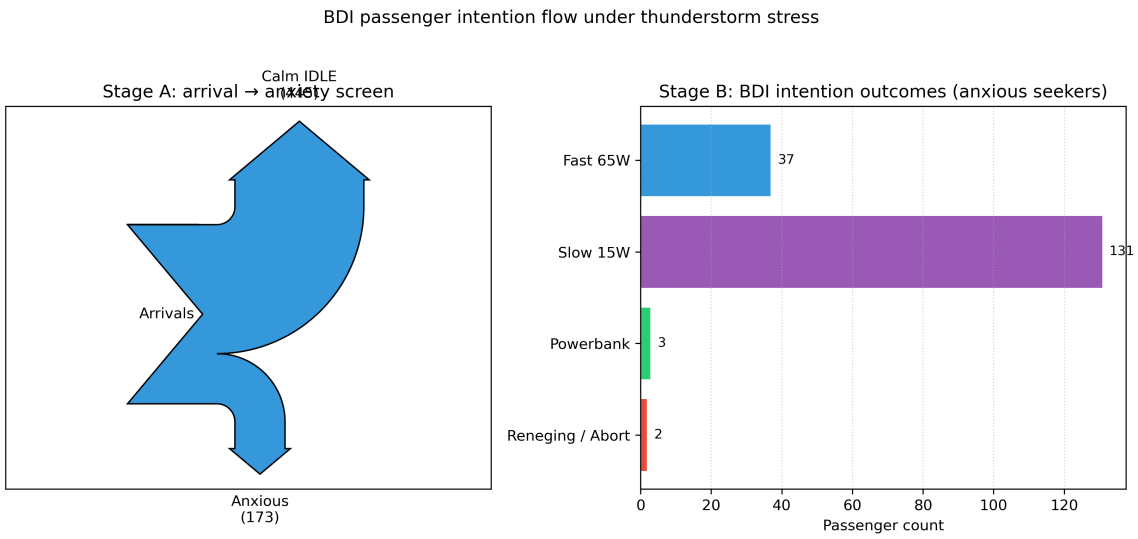


Figure 4: BDI passenger agent intention flow under storm stress. Source: `fig_bdi_intention_flow_sankey.png`.

Operational reading. More fast kiosks alone do not buy resilience. The knee keeps sixteen slow lockers and zero power-bank banks. Under this seed, slow capacity absorbs stranded dwell better than the heavier empirical mix.

6 Strengths and limitations

Strengths.

1. Every KPI cites a locked CSV path under `results/`.
2. BDI passenger agents couple SOC, boarding slack, and reneging.
3. FastMCP + LangGraph make DES calls tool-shaped and auditable.
4. Pareto filter enforces reneging probability ≤ 0.05.
5. Storm test uses one seed for fair pack comparison.

Limitations.

1. Arrival intensity is synthetic NHPP, not airport AODB feeds.
2. Gate geography is not a spatial queueing network.
3. Power-bank banks are zero on the current knee; inventory risk is not priced.
4. Ollama drafts proposals; offline mode falls back to rules.
5. One storm seed is not a full Monte Carlo envelope.

Recommendation. Deploy pack 8/16/0 first (\$35,200, 38.4 m^2). Keep CapEx headroom under \$80,000 for gate-cluster add-ons. Re-run storm seeds before capital release.

References

References

[1] BDI–DES passenger decision model (arXiv:2602.13878). Local PDF: `data/raw/papers/arxiv_2602.13878_bdi_des.pdf`.

[2] Team SimPy. *SimPy: Discrete-event simulation in Python.* `https://simpy.readthedocs.io/`.

[3] Anthropic. *Model Context Protocol (MCP) specification.* `https://modelcontextprotocol.io/`.

[4] LangChain. *LangGraph.* `https://langchain-ai.github.io/langgraph/`.